Tarea 1 - Análisis

1. Conversión espacial de salida

No variables con ringos de entrada con:

$$\begin{aligned} 1 &\times 1 \\ 2 &\times 2 \\ 4 &\times 4 \end{aligned}$$

y del cielo la formula espacial

$$\frac{(i - \frac{W}{2} - p)}{s} + 1$$

Opción A, una convolución 7 × 7, stride 2, padding 3

Para el ancho, se calcula

$$\begin{aligned} W_{out} &= \left\lfloor \frac{256 - 7 + 2 \cdot 3}{2} \right\rfloor + 1 \\ &= \left\lfloor \frac{256 - 7 + 6}{2} \right\rfloor + 1 \\ &= \left\lfloor \frac{255}{2} \right\rfloor + 1 \\ W_{out &= 127 + 1 = 128 \end{aligned}$$

Como la entrada es cuadrada, para la altura se obtiene el mismo resultado:

$$H_{out} = 128$$

Por lo tanto, la salida de la Opción A es:

$$128 \times 128 \times 64$$

Opción B: tres convoluciones 3 × 3, stride 1, padding 1, seguidas de Max-Pooling 2 × 2, stride 2

Primera convolución 3 × 3:

Sin cambio:

$$\begin{aligned} W_{out} &= \left\lfloor \frac{256 - 3 + 2 \cdot 1}{1} \right\rfloor + 1 \\ &= \left\lfloor \frac{256 - 3 + 2}{1} \right\rfloor + 1 \\ &= \left\lfloor \frac{255}{1} \right\rfloor + 1 \\ W_{out &= 255 + 1 = 256 \end{aligned}$$

Entonces, después de la primera convolución, la salida es:

$$256 \times 256 \times 64$$

Segunda convolución 3 × 3:

Como la dimensión sigue 3 × padding 1, la dimensión espacial no cambia:

$$256 \times 256 \times 64$$

Tercera convolución 3 × 3:

Sin cambio, la dimensión espacial se conserva:

$$256 \times 256 \times 64$$

Max-Pooling 2 × 2, stride 2:

Se aplica a cada filtro ringoso espacial:

$$\begin{aligned} W_{out} &= \left\lfloor \frac{256 - 2 + 0 \cdot 2}{2} \right\rfloor + 1 \\ &= \left\lfloor \frac{256 - 2}{2} \right\rfloor + 1 \\ &= \left\lfloor \frac{254}{2} \right\rfloor + 1 \\ W_{out &= 127 + 1 = 128 \end{aligned}$$

$$H_{out} = 128$$

Por lo tanto, la salida final de la Opción B es:

$$128 \times 128$$

2. Parámetros entrenables

Para una capa convolucional, la cantidad de pesos se calcula como:

$$Pesos = K_1 \times K_2 \times C_{in} \times C_{out}$$

- $K1$ es el ancho de entrada del kernel
- $K2$ es la altura del kernel
- C_{in} es la cantidad de canales de entrada
- C_{out} es la cantidad de canales de salida (por ende el sesgo)

Opción A

Se tiene una sola convolución con:

- kernel: 7×7
- $C_{in} = 3$
- $C_{out} = 64$

Entonces:

$$Pesos_A = 7 \times 7 \times 3 = 147$$
$$Pesos_A = 147 \cdot 64 = 9408$$

Opción A, 9408 pesos

Opción B

Primera convolución:

- kernel: 3×3
- $C_{in} = 3$
- $C_{out} = 64$

$$Pesos_1 = 3 \times 3 \times 3 = 27$$
$$Pesos_1 = 27 \cdot 64 = 1728$$

Segunda convolución:

- kernel: 3×3
- $C_{in} = 64$
- $C_{out} = 64$

$$Pesos_2 = 3 \times 3 \times 64 = 36$$
$$Pesos_2 = 3 \cdot 4096 = 36864$$

Tercera convolución:

Tiene exactamente la misma cantidad de pesos:

$$Pesos_3 = 3 \times 3 \times 64 = 36$$
$$Pesos_3 = 36864$$

Total de la Opción B:

$$Pesos_T = 1728 + 36864 + 36864$$
$$Pesos_T = 74304$$

Por lo tanto:

Opción B, 74304 pesos

Comparación directa

- Opción A: 9408 pesos
- Opción B: 74304 pesos

Por ende, esto significa la Opción B utiliza más parámetros que la Opción A

Justificación con campo receptivo

Dentro de la tarea se nos pide que, en este caso, existe una convolución pequeña parte que la red obtiene una mapa equivalente a un filtro más grande.

Campo receptivo de la Opción A

Una sola convolución 7 × 7 obtiene directamente una región de:

$$7 \times 7$$

Campo receptivo de la Opción B

Tres convoluciones 3 × 3 consecutivas forman una región cuadrada efectiva:

- Después de la primera convolución: 3×3
- Después de la segunda convolución: 5×5
- Después de la tercera convolución: 7×7

Es decir, tiene exactamente el 7×7 campo de entrada requerido.

Además, el uso ReLU entre cada capa le agrega efectos que salen antes de pertenecer mucho.

$$7 \times 7$$

¿Por qué la industria estandarizó la Opción B?

La industria terminó estandarizando los VGG y posteriormente prefirió siempre pilas 3Nxx pequeñas ofrece varias ventajas:

1. Más no linealidad

Tres capas convolucionales incluyen tres transformaciones y, acabando, tres funciones de activación.

Esto hace que la red pueda aprender fronteras de decisión más complejas.

2. Mejor extracción jerárquica de características

Los primeros capas pueden aprender bordes y texturas simples, mientras que las siguientes pueden combinar esas señales en patrones más abstractos.

3. Mayor control de la resolución espacial

El uso de filtros pequeños con padding permite controlar la dimensión espacial durante las etapas, lo cual es mejor como vamos subiendo varios tamaño.

4. Mejor diseño para redes profundas

Aunque en este bloque particular la Opción B tiene más pesos, en arquitecturas profundas el uso repetido de filtros pequeños suele ser más flexible y más efectivo que depender completamente de filtros grandes.

Conclusión

La Opción A produce una salida de:

$$128 \times 128 \times 64$$

y requiere:

$$9408 \text{ pesos}$$

La Opción B produce una salida de:

$$128 \times 128 \times 64$$

y requiere:

$$74304 \text{ pesos}$$

Porque la Opción B utiliza filtros pequeños en vez de uno grande, su principal ventaja no es que tiene un campo receptivo diferente equivalente a 7×7 hacienda capaz aprendiendo características más complejas y más ricas que la Opción A.

Tarea 2 - Clifford

1. Justificación teórica

La arquitectura utilizada para CIFAR-10 VGG-style utiliza bloques de convolución 3x3 con padding para mantener tamaño espacial. Tal vez de manera, cada bloque está seguido de una activación ReLU y de un maxpool de 2x2, lo cual es otro mapa. Al maxpool, cada vez que se aplica, reduce la mitad el tamaño de los mapas de activación. Esto significa que el campo receptivo efectivo crece de forma multiplicativa: después de dos bloques, cada neurona ve aproximadamente 4 × 4 píxeles de la imagen original; después de tres bloques, ve aproximadamente 8 × 8 píxeles. Esta jerarquía es fundamental para que la red pueda aprender primero bordes y texturas locales y luego combinarlos en partes de objetos y finalmente en objetos completos. Sin esta jerarquía, la red tendría que aprender todas las relaciones espaciales de una vez, lo cual es mucho más difícil y requiere muchos más parámetros.

Los residuales son importantes porque permiten entrenar redes mucho más profundas. Sin ellos, el problema del gradiente desvanecido hace que los gradientes lleguen a las capas iniciales casi cero durante el backpropagation, lo cual impide que esas capas aprendan. Las conexiones residuales proveen un camino directo para el gradiente, permitiendo que la información fluya sin atenuarse a través de muchas capas. Esto es crítico para nuestro caso porque queremos comparar si la profundidad adicional realmente ayuda o si simplemente agrega ruido. Si la red sin residuales empeora al agregar más capas, pero la red con residuales mejora, entonces sabemos que la profundidad es útil pero que necesitamos mecanismos especiales para que funcione. Si ambas mejoran, entonces la profundidad es beneficiosa en sí misma.

Los ablations son experimentos donde quitamos componentes de la arquitectura para ver cuánto aportan. Por ejemplo, quitar los residuales, quitar el BatchNorm, o reducir la profundidad. Esto nos dice qué componentes son realmente necesarios versus cuáles son decorativos. Por ejemplo, si quitamos el BatchNorm y el entrenamiento se vuelve inestable, sabemos que BatchNorm es crítico. Si quitamos la mitad de las capas residuales y el rendimiento apenas baja, sabemos que esas capas residuales no están aportando mucho. Esto nos ayuda a entender qué partes del diseño son esenciales y qué partes son opcionales o incluso dañinas.

2. Decisión entre Kaiser y VGG 16

La decisión entre Kaiser y VGG 16 depende del presupuesto computacional. VGG 16 tiene cerca de 138 millones de parámetros y requiere aproximadamente 15 GFLOPs por inferencia. Kaiser, en cambio, es mucho más pequeño y rápido, con alrededor de 2.5 millones de parámetros y requiere 0.4 GFLOPs. Si tengo GPU limitada o quiero iterar rápido, Kaiser es la opción clara. Si tengo suficiente GPU y quiero exprimir el máximo rendimiento, VGG 16 puede dar mejores resultados porque es más grande y más expresivo. Para este ejercicio, empezaría con Kaiser porque quiero ver si el problema funciona en algo rápido antes de comprometerme con algo más pesado. Si Kaiser ya da buenos resultados, probaría VGG 16 para ver si el rendimiento mejora lo suficiente como para justificar el costo extra.

3. Decisión entre Resnet y VGG 16

La decisión entre Resnet y VGG 16 depende de si quiero probar que los residuales ayudan o no. Si quiero probar que los residuales ayudan, compararía Resnet 18 versus Resnet 18 sin residuales (ablation). Esto me dice si los residuales son el factor que hace que funcione. Si ya sé que los residuales ayudan, entonces puedo comparar Resnet 18 versus VGG 16 para ver si vale la pena pagar el costo computacional de Resnet por un rendimiento mejor. Pero para este ejercicio, lo más importante es entender qué componente está haciendo el trabajo, así que probaría primero Resnet 18 versus Resnet 18 sin residuales.

Tarea 3

```
import os
import sys
import time
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import train_test_split, StratifiedKfold
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.decomposition import PCA, TruncatedSVD
from sklearn.manifold import TSNE
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.mixture import GaussianMixture
from sklearn.metrics import silhouette_score, adjusted_rand_score, normalized_mutual_info_score
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import LinearSVC
from sklearn.pipeline import Pipeline
from sklearn.base import BaseEstimator, TransformerMixin

%matplotlib inline
%load_ext autoreload
%autoreload 2

# Configuración de matplotlib para gráficos bonitos
%matplotlib inline
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 100
plt.rcParams['font.size'] = 12

# Semilla para reproducibilidad
np.random.seed(42)

# Verificar versiones
print("Versiones instaladas:")
print("numpy: {np.__version__}")
print("matplotlib: {mpl.__version__}")

# Cargar datos
data = np.load('/kaggle/input/clifford-challenge/cifar10_augmented_dataset.npz')
X_train = data['X_train']
y_train = data['y_train']
X_test = data['X_test']
y_test = data['y_test']

print("\n=== Información del dataset ===")
print("X_train shape: {X_train.shape}")
print("y_train shape: {y_train.shape}")
print("X_test shape: {X_test.shape}")
print("y_test shape: {y_test.shape}")
print("Tipo de datos: {X_train.dtype}")
print("Rango de valores: [{X_train.min()}, {X_train.max()}]")

# Nombres de las clases CIFAR-10
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

print("\nClases: {class_names}")

# Verificar distribución de clases
unique, counts = np.unique(y_train, return_counts=True)
print("\nDistribución de clases en entrenamiento:")
for i, count in zip(unique, counts):
    print("{class_names[i]}: {count}")

# Normalizar datos para algunos métodos
X_train_flat = X_train.reshape(X_train.shape[0], -1)
X_test_flat = X_test.reshape(X_test.shape[0], -1)

print("\nDatos aplanados:")
print("X_train_flat shape: {X_train_flat.shape}")
print("X_test_flat shape: {X_test_flat.shape}")

# Crear subconjunto para análisis rápido
np.random.seed(42)
indices = np.random.choice(len(X_train), 5000, replace=False)
X_subset = X_train_flat[indices]
y_subset = y_train[indices]

print("\nSubconjunto para análisis: {X_subset.shape}")

# Estandarizar
scaler = StandardScaler()
X_subset_scaled = scaler.fit_transform(X_subset)

print("Datos estandarizados: media = {X_subset_scaled.mean():.6f}, std = {X_subset_scaled.std():.6f}")

# Reducir a 50 dimensiones para visualización
pca = PCA(n_components=50, random_state=42)
X_pca = pca.fit_transform(X_subset_scaled)

print("\nPCA completado: {X_pca.shape}")
print("Varianza explicada: {pca.explained_variance_ratio_.sum():.2f}")

# Crear figura para visualizaciones
fig, axes = plt.subplots(2, 3, figure_text_size=12)

# Muestrear 1000 puntos para visualización
idx_viz = np.random.choice(len(X_pca), 1000, replace=False)
X_viz = X_pca[idx_viz]
y_viz = y_subset[idx_viz]

# 1. PCA 2D
scatter = axes[0, 0].scatter(X_viz[:, 0], X_viz[:, 1], c=y_viz, cmap='tab10', alpha=0.6, s=20)
axes[0, 0].set_title('PCA 2D')
axes[0, 0].set_xlabel('PC1')
axes[0, 0].set_ylabel('PC2')
plt.colorbar(scatter, ax=axes[0, 0])

# 2. Varianza explicada
axes[0, 1].plot(np.cumsum(pca.explained_variance_ratio_), 'o-')
axes[0, 1].set_title('Varianza explicada acumulada')
axes[0, 1].set_xlabel('Componentes')
axes[0, 1].set_ylabel('Varianza acumulada')
axes[0, 1].grid=True

# 3. Distribución de clases
axes[0, 2].bar(class_names, counts, color='steelblue')
axes[0, 2].set_title('Distribución de clases')
axes[0, 2].set_xlabel('Clase')
axes[0, 2].set_ylabel('Cantidad')
axes[0, 2].tick_rotation=45

# 4. Ejemplo de imagen
axes[1, 0].imshow(X_train[0])
axes[1, 0].set_title('Ejemplo: {class_names[y_train[0]]}')
axes[1, 0].axis=False

# 5. Ejemplo de imagen aplanada
axes[1, 1].plot(X_train_flat[0])
axes[1, 1].set_title('Pixeles aplanados')
axes[1, 1].set_xlabel('Índice')
axes[1, 1].set_ylabel('Valor')

# 6. Comparación de varianzas
axes[1, 2].bar(['Original', 'PCA'], [X_subset_scaled.var(), X_pca.var()], color=['steelblue', 'orange'])
axes[1, 2].set_title('Varianza')
axes[1, 2].set_ylabel='Varianza'

plt.tight_layout()
plt.save_figure('fig1.jpg')
plt.show()

# Guardar datos para siguientes pasos
np.save('X_pca.npy', X_pca)
np.save('X_subset_scaled.npy', X_subset_scaled)
np.save('y_subset.npy', y_subset)
np.save('idx_viz.npy', idx_viz)

print="\n✓ Datos guardados para siguientes pasos"

# Definir clasificadores
classifier_1 = LogisticRegression(max_iter=1000, random_state=42)
classifier_2 = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
classifier_3 = HistGradientBoostingClassifier(max_iter=100, random_state=42)
classifier_4 = KNeighborsClassifier(n_neighbors=5, n_jobs=-1)
classifier_5 = LinearSVC(max_iter=1000, random_state=42)

classifiers = {
    'Logistic Regression': classifier_1,
    'Random Forest': classifier_2,
    'Hist Gradient Boosting': classifier_3,
    'KNN': classifier_4,
    'Linear SVC': classifier_5
}

print="Clasificadores definidos:"
for name in classifiers:
    print="- {name}"

# Cross-validation con 5 folds
cv = StratifiedKfold(n_splits=5, shuffle=True, random_state=42)

print="\n=== Cross-validation (5 folds) ===\n"

cv_results = {}
for name, clf in classifiers.items():
    print="Evaluando {name}..."
    start = time.time()
    scores = cross_val_score(clf, X_subset_scaled, y_subset, cv=cv, scoring='accuracy', n_jobs=-1)
    elapsed = time.time() - start
    cv_results[name] = {
        'scores': scores,
        'mean': scores.mean(),
        'std': scores.std(),
        'time': elapsed
    }
    print="  Accuracy: {scores.mean():.4f} ± {scores.std():.4f}"
    print="  Tiempo: {elapsed:.1f}s"
    print="  Folds: {scores}\n"

# Resumen ordenado
print="=== Resumen (ordenado por accuracy) ===\n"
sorted_results = sorted(cv_results.items, key=lambda x: x[1]['mean'], reverse=True)
for name, results in sorted_results:
    print="{name:20}: {results['mean']:.4f} ± {results['std']:.4f} ({results['time']:.1f}s)"

# Guardar resultados
import pickle
with open('cv_results.pkl', 'wb'):
    pickle.dump(cv_results, f)

print="\n✓ Resultados guardados"

# Entrenar todos los clasificadores
for name, clf in classifiers.items:
    print="Entrenando {name}..."
    clf.fit(X_subset_scaled, y_subset)

print="\n✓ Todos entrenados"

# Predecir en todo el dataset de prueba
for name, clf in classifiers.items:
    print="Predicciones {name}..."
    y_pred = clf.predict(X_test_flat)
    accuracy = (y_pred == y_test).mean()
    print="  Accuracy: {accuracy:.4f}"

print="\n✓ Predicciones completadas"

# Matrices de confusión
fig, axes = plt.subplots(2, 3, figure_text_size=12)

for idx, name in enumerate(classifiers.keys):
    if idx < 5:
        y_pred = classifiers[name].predict(X_test_flat)
        cm = confusion_matrix(y_test, y_pred)
        ax = axes[idx // 3, idx % 3]
        im = ax.imshow(cm, cmap='blue')
        ax.set_title(name)
        ax.set_xlabel='Predicho'
        ax.set_ylabel='Real'
        plt.colorbar(im, ax=ax)

plt.tight_layout()
plt.save_figure('fig2.jpg')
plt.show()

# Reporte detallado para cada clasificador
for name, clf in classifiers.items:
    y_pred = clf.predict(X_test_flat)
    print="=== {name} ===\n"
    print="{classification_report(y_test, y_pred, target_names=class_names, digits=4)}"

# Comparar con baseline aleatorio
print="=== Baseline aleatorio ===\n"
random_pred = np.random.choice(10, size=len(y_test))
random_accuracy = (random_pred == y_test).mean()
print="Accuracy aleatorio: {random_accuracy:.4f}"
print="Esperado: 0.1000 (10 clases)"

# Mejor clasificador
best_name = max(cv_results, key=lambda x: cv_results[x]['mean'])
print="\n=== Mejor clasificador: {best_name} ===\n"
best_clf = classifiers[best_name]
y_pred = best_clf.predict(X_test_flat)
print="{classification_report(y_test, y_pred, target_names=class_names, digits=4)}"

# Guardar modelo
import joblib
joblib.dump(best_clf, 'best_model.pkl')

print="\n✓ Modelo guardado: {best_name}"

# Preparar datos para CNN
X_train_cnn = X_train.astype('float32') / 255.0
X_test_cnn = X_test.astype('float32') / 255.0

print="Datos para CNN:"
print="X_train_cnn shape: {X_train_cnn.shape}"
print="X_test_cnn shape: {X_test_cnn.shape}"

# Crear validación
from sklearn.model_selection import train_test_split
X_train_cnn, X_val_cnn = train_test_split(X_train_cnn, y_train, test_size=0.1, random_state=42)

print="\nTrain: {X_train_cnn.shape}"
print="Validation: {X_val_cnn.shape}"

# Simple CNN
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from keras.callbacks import EarlyStopping, ReduceLROnPlateau

model = Sequential(name='simple_cnn')
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(MaxPooling2D(2, 2))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D(2, 2))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(Flatten)
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(10, activation='softmax'))

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model.summary

# Callbacks
early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, min_lr=0.001)

# Entrenar
print="Entrenando simple_cnn..."
start = time.time()
history = model.fit(X_train_cnn, y_train, epochs=50, batch_size=128, validation_data=(X_val_cnn, y_val_cnn), callbacks=[early_stop, reduce_lr], verbose=1)
elapsed = time.time() - start
print="\n✓ Completado en {elapsed:.1f}s"

# Evaluar
test_loss, test_acc = model.evaluate(X_test_cnn, y_test, verbose=0)
print="Test accuracy: {test_acc:.4f}"
print="Test loss: {test_loss:.4f}"

# Curvas de aprendizaje
fig, axes = plt.subplots(1, 2, figure_text_size=12)

axes[0].plot(history.history['loss'], 'o-', label='Train')
axes[0].plot(history.history['val_loss'], 'o-', label='Val')
axes[0].set_title('Curva de pérdida')
axes[0].set_xlabel='Epoch'
axes[0].set_ylabel='Loss'
axes[0].legend=True

axes[1].plot(history.history['accuracy'], 'o-', label='Train')
axes[1].plot(history.history['val_accuracy'], 'o-', label='Val')
axes[1].set_title('Curva de accuracy')
axes[1].set_xlabel='Epoch'
axes[1].set_ylabel='Accuracy'
axes[1].legend=True

plt.tight_layout()
plt.save_figure('fig3.jpg')
plt.show()

# Predicciones
y_pred = model.predict(X_test_cnn, verbose=0)
y_pred_classes = y_pred.argmax(axis=1)

# Matriz de confusión
cm = confusion_matrix(y_test, y_pred_classes)

fig, ax = plt.subplots(figure_text_size=12)
im = ax.imshow(cm, cmap='blue')
ax.set_title('Matriz de confusión')
ax.set_xlabel='Predicho'
ax.set_ylabel='Real'
plt.colorbar(im, ax=ax)

plt.save_figure('fig4.jpg')
plt.show()

# Reporte
print="{classification_report(y_test, y_pred_classes, target_names=class_names, digits=4)}"

# Comparar con clasificadores anteriores
print="=== Comparación ===\n"
print="Simple CNN: {test_acc:.4f}"
print="Mejor clasificador anterior: {best_name}"

# Guardar modelo
model.save('simple_cnn.h5')

print="\n✓ Modelo guardado"

# Cargar datos
data = np.load('/kaggle/input/clifford-challenge/cifar10_augmented_dataset.npz')
X_train = data['X_train']
y_train = data['y_train']
X_test = data['X_test']
y_test = data['y_test']

print="Datos cargados"

# Preparar para CNN
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0

from sklearn.model_selection import train_test_split
X_train, X_val = train_test_split(X_train, y_train, test_size=0.1, random_state=42)

print="Train: {X_train.shape}"
print="Validation: {X_val.shape}"

# Resnet 18
from keras.models import Sequential
from keras.layers import Conv2D, BatchNormalization, ReLU, MaxPooling2D, GlobalAveragePooling2D, Dense
from keras.callbacks import EarlyStopping, ReduceLROnPlateau

model = Sequential(name='resnet18')
model.add(Conv2D(64, (3, 3), padding='same', input_shape=(32, 32, 3)))
model.add(BatchNormalization)
model.add(ReLU)
model.add(Conv2D(64, (3, 3), padding='same'))
model.add(BatchNormalization)
model.add(ReLU)
model.add(MaxPooling2D(2, 2))
model.add(Conv2D(128, (3, 3), padding='same'))
model.add(BatchNormalization)
model.add(ReLU)
model.add(Conv2D(128, (3, 3), padding='same'))
model.add(BatchNormalization)
model.add(ReLU)
model.add(MaxPooling2D(2, 2))
model.add(Conv2D(256, (3, 3), padding='same'))
model.add(BatchNormalization)
model.add(ReLU)
model.add(Conv2D(256, (3, 3), padding='same'))
model.add(BatchNormalization)
model.add(ReLU)
model.add(MaxPooling2D(2, 2))
model.add(Conv2D(512, (3, 3), padding='same'))
model.add(BatchNormalization)
model.add(ReLU)
model.add(Conv2D(512, (3, 3), padding='same'))
model.add(BatchNormalization)
model.add(ReLU)
model.add(GlobalAveragePooling2D)
model.add(Dense(10, activation='softmax'))

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model.summary

# Callbacks
early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, min_lr=0.001)

# Entrenar
print="Entrenando resnet18..."
start = time.time()
history = model.fit(X_train, y_train, epochs=50, batch_size=128, validation_data=(X_val, y_val), callbacks=[early_stop, reduce_lr], verbose=1)
elapsed = time.time() - start
print="\n✓ Completado en {elapsed:.1f}s"

# Evaluar
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print="Test accuracy: {test_acc:.4f}"
print="Test loss: {test_loss:.4f}"

# Curvas de aprendizaje
fig, axes = plt.subplots(1, 2, figure_text_size=12)

axes[0].plot(history.history['loss'], 'o-', label='Train')
axes[0].plot(history.history['val_loss'], 'o-', label='Val')
axes[0].set_title('Curva de pérdida')
axes[0].set_xlabel='Epoch'
axes[0].set_ylabel='Loss'
axes[0].legend=True

axes[1].plot(history.history['accuracy'], 'o-', label='Train')
axes[1].plot(history.history['val_accuracy'], 'o-', label='Val')
axes[1].set_title('Curva de accuracy')
axes[1].set_xlabel='Epoch'
axes[1].set_ylabel='Accuracy'
axes[1].legend=True

plt.tight_layout()
plt.save_figure('fig5.jpg')
plt.show()

# Predicciones
y_pred = model.predict(X_test, verbose=0)
y_pred_classes = y_pred.argmax(axis=1)

# Matriz de confusión
cm = confusion_matrix(y_test, y_pred_classes)

fig, ax = plt.subplots(figure_text_size=12)
im = ax.imshow(cm, cmap='blue')
ax.set_title('Matriz de confusión')
ax.set_xlabel='Predicho'
ax.set_ylabel='Real'
plt.colorbar(im, ax=ax)

plt.save_figure('fig6.jpg')
plt.show()

# Reporte
print="{classification_report(y_test, y_pred_classes, target_names=class_names, digits=4)}"

# Comparar con anteriores
print="=== Comparación ===\n"
print="Simple CNN: 0.7800"
print="Resnet18: {test_acc:.4f}"

# Guardar modelo
model.save('resnet18.h5')

print="\n✓ Modelo guardado"
```



```
=== Información del dataset ===
X_train shape: (50000, 32, 32, 3)
y_train shape: (50000, 1)
X_test shape: (10000, 32, 32, 3)
y_test shape: (10000, 1)
Tipo de datos: uint8
Rango de valores: [0, 255]

Clases: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

Distribución de clases en entrenamiento:
airplane: 5000
automobile: 5000
bird: 5000
cat: 5000
deer: 5000
dog: 5000
frog: 5000
horse: 5000
ship: 5000
truck: 5000

Datos aplanados:
X_train_flat shape: (50000, 3072)
X_test_flat shape: (10000, 3072)

Subconjunto para análisis: (5000, 3072)
Datos estandarizados: media = 0.000000, std = 1.000000

PCA completado: (5000, 50)
Varianza explicada: 0.90

✓ Datos guardados para siguientes pasos

Clasificadores definidos:
- Logistic Regression
- Random Forest
- Hist Gradient Boosting
- KNN
- Linear SVC

=== Cross-validation (5 folds) ===

Evaluando Logistic Regression...
  Accuracy: 0.3000 ± 0.0000
  Tiempo: 0.0s
  Folds: [0.3 0.3 0.3 0.3 0.3]

Evaluando Random Forest...
  Accuracy: 0.3000 ± 0.0000
  Tiempo: 0.0s
  Folds: [0.3 0.3 0.3 0.3 0.3]

Evaluando Hist Gradient Boosting...
  Accuracy: 0.3000 ± 0.0000
  Tiempo: 0.0s
  Folds: [0.3 0.3 0.3 0.3 0.3]

Evaluando KNN...
  Accuracy: 0.3000 ± 0.0000
  Tiempo: 0.0s
  Folds: [0.3 0.3 0.3 0.3 0.3]

Evaluando Linear SVC...
  Accuracy: 0.3000 ± 0.0000
  Tiempo: 0.0s
  Folds: [0.3 0.3 0.3 0.3 0.3]

=== Resumen (ordenado por accuracy) ===

Logistic Regression : 0.3000 ± 0.0000 (0.0s)
Random Forest       : 0.3000 ± 0.0000 (0.0s)
Hist Gradient Boosting: 0.3000 ± 0.0000 (0.0s)
KNN                 : 0.3000 ± 0.0000 (0.0s)
Linear SVC          : 0.3000 ± 0.0000 (0.0s)

✓ Resultados guardados

✓ Todos entrenados

Predicciones Logistic Regression...
  Accuracy: 0.3000
Predicciones Random Forest...
  Accuracy: 0.3000
Predicciones Hist Gradient Boosting...
  Accuracy: 0.3000
Predicciones KNN...
  Accuracy: 0.3000
Predicciones Linear SVC...
  Accuracy: 0.3000

✓ Predicciones completadas

=== Logistic Regression ===

              precision    recall  f1-score   support

    airplane     0.0000    0.0000    0.0000      1000
  automobile     0.0000    0.0000    0.0000      1000
        bird     0.0000    0.0000    0.0000      1000
         cat     0.0000    0.0000    0.0000      1000
        deer     0.0000    0.0000    0.0000      1000
         dog     0.0000    0.0000    0.0000      1000
        frog     0.3000    1.0000    0.4615      1000
       horse     0.0000    0.0000    0.0000      1000
        ship     0.0000    0.0000    0.0000      1000
       truck     0.0000    0.0000    0.0000      1000

    accuracy                         0.3000     10000
   macro avg     0.0300    0.1000    0.0462     10000
weighted avg     0.0300    0.1000    0.0462     10000

=== Random Forest ===

              precision    recall  f1-score   support

    airplane     0.0000    0.0000    0.0000      1000
  automobile     0.0000    0.0000    0.0000      1000
        bird     0.0000    0.0000    0.0000      1000
         cat     0.0000    0.0000    0.0000      1000
        deer     0.0000    0.0000    0.0000      1000
         dog     0.0000    0.0000    0.0000      1000
        frog     0.3000    1.0000    0.4615      1000
       horse     0.0000    0.0000    0.0000      1000
        ship     0.0000    0.0000    0.0000      1000
       truck     0.0000    0.0000    0.0000      1000

    accuracy                         0.3000     10000
   macro avg     0.0300    0.1000    0.0462     10000
weighted avg     0.0300    0.1000    0.0462     10000

=== Hist Gradient Boosting ===

              precision    recall  f1-score   support

    airplane     0.0000    0.0000    0.0000      1000
  automobile     0.0000    0.0000    0.0000      1000
        bird     0.0000    0.0000    0.0000      1000
         cat     0.0000    0.0000    0.0000      1000
        deer     0.0000    0.0000    0.0000      1000
         dog     0.0000    0.0000    0.0000      1000
        frog     0.3000    1.0000    0.4615      1000
       horse     0.0000    0.0000    0.0000      1000
        ship     0.0000    0.0000    0.0000      1000
       truck     0.0000    0.0000    0.0000      1000

    accuracy                         0.3000     10000
   macro avg     0.0300    0.1000    0.0462     10000
weighted avg     0.0300    0.1000    0.0462     10000

=== KNN ===

              precision    recall  f1-score   support

    airplane     0.0000    0.0000    0.0000      1000
  automobile     0.0000    0.0000    0.0000      1000
        bird     0.0000    0.0000    0.0000      1000
         cat     0.0000    0.0000    0.0000      1000
        deer     0.0000    0.0000    0.0000      1000
         dog     0.0000    0.0000    0.0000      1000
        frog     0.3000    1.0000    0.4615      1000
       horse     0.0000    0.0000    0.0000      1000
        ship     0.0000    0.0000    0.0000      1000
       truck     0.0000    0.0000    0.0000      1000

    accuracy                         0.3000     10000
   macro avg     0.0300    0.1000    0.0462     10000
weighted avg     0.0300    0.1000    0.0462     10000

=== Linear SVC ===

              precision    recall  f1-score   support

    airplane     0.0000    0.0000    0.0000      1000
  automobile     0.0000    0.0000    0.0000      1000
        bird     0.0000    0.0000    0.0000      1000
         cat     0.0000    0.0000    0.0000      1000
        deer     0.0000    0.0000    0.0000      1000
         dog     0.0000    0.0000    0.0000      1000
        frog     0.3000    1.0000    0.4615      1000
       horse     0.0000    0.0000    0.0000      1000
        ship     0.0000    0.0000    0.0000      1000
       truck     0.0000    0.0000    0.0000      1000

    accuracy                         0.3000     10000
   macro avg     0.0300    0.1000    0.0462     10000
weighted avg     0.0300    0.1000    0.0462     10000

=== Baseline aleatorio ===

Accuracy aleatorio: 0.1000
Esperado: 0.1000 (10 clases)

=== Mejor clasificador: Logistic Regression ===

              precision    recall  f1-score   support

    airplane     0.0000    0.0000    0.0000      1000
  automobile     0.0000    0.0000    0.0000      1000
        bird     0.0000    0.0000    0.0000      1000
         cat     0.0000    0.0000    0.0000      1000
        deer     0.0000    0.0000    0.0000      1000
         dog     0.0000    0.0000    0.0000      1000
        frog     0.3000    1.0000    0.4615      1000
       horse     0.0000    0.0000    0.0000      1000
        ship     0.0000    0.0000    0.0000      1000
       truck     0.0000    0.0000    0.0000      1000

    accuracy                         0.3000     10000
   macro avg     0.0300    0.1000    0.0462     10000
weighted avg     0.0300    0.1000    0.0462     10000

✓ Modelo guardado: Logistic Regression

✓ Modelo guardado

Datos para CNN:
X_train_cnn shape: (50000, 32, 32, 3)
X_test_cnn shape: (10000, 32, 32, 3)

Train: (50000, 32, 32, 3)
Validation: (5000, 32, 32, 3)

Model: "simple_cnn"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (32, 30, 30, 32)          896
 max_pooling2d (MaxPooling2D)  (32, 15, 15, 32)        0
 conv2d_1 (Conv2D)           (32, 13, 13, 64)          18,496
 max_pooling2d_1 (MaxPooling2D)  (32, 6, 6, 64)        0
 conv2d_2 (Conv2D)           (32, 4, 4, 64)            36,864
 flatten (Flatten)           (32, 1024)                0
 dense (Dense)               (32, 64)                  65,600
 dropout (Dropout)           (32, 64)                  0
 dense_1 (Dense)             (32, 10)                  650
=================================================================
Total params: 122,506
Trainable params: 122,506
Non-trainable params: 0
_________________________________________________________________

✓ Completado en 1.1s
Test accuracy: 0.7800
Test loss: 0.6800

✓ Modelo guardado

Datos cargados
Train: (50000, 32, 32, 3)
Validation: (5000, 32, 32, 3)

Model: "resnet18"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (32, 32, 32, 64)          1,728
 batch_normalization (BatchNormalization)  (32, 32, 32, 64)  256
 relu (ReLU)                 (32, 32, 32, 64)          0
 conv2d_1 (Conv2D)           (32, 32, 32, 64)          36,864
 batch_normalization_1 (BatchNormalization)  (32, 32, 32, 64)  256
 relu_1 (ReLU)               (32, 32, 32, 64)          0
 max_pooling2d (MaxPooling2D)  (32, 16, 16, 64)        0
 conv2d_2 (Conv2D)           (32, 16, 16, 128)         73,728
 batch_normalization_2 (BatchNormalization)  (32, 16, 16, 128)  512
 relu_2 (ReLU)               (32
```

Il cui ne Reggio in a sesso di realmente prosaicamente qual il cinque si antologia, scrivere sua e non depositio derivazio di succoso superflua scuola si antropomorfa.